Module 10

Recursion: Calling yourself again

Last time:

When you define your own objects (types) and add them into a data structure and then sort them, you need to make sure that they are mutually comparable. In Java, this can be accomplished by implementing either *Comparable* interface or *Comparator* interface. In some (or many) cases, both!

- Your class can implement the *Comparable* interface. In that case, you are to implement `compareTo(T e)` method in your class.
- You can also create separate (nested) classes that implement the *Comparator* interface. You need to then implement `compare(T o1, T o2)` method.
- This enables you to have different ways of comparing objects.

It is important to remember that there is a strong relationship between `compareTo()` and `equals()` methods.

> **If `x.equals(y)` is true, then `x.compareTo(y) == 0`.**

Additionally, if the `equals()` method is overridden, then the `hashCode()` method also needs to be taken care of accordingly.

> **If `x.equals(y)` is true, then `x.hashCode() == y.hashCode()`.**

This time, let's explore a different way of thinking, called Recursion.

Adding positive integers from 1 to n

1 + 2 + 3 + 4 + … + n

How would you solve this problem in Java?

```
public static int sum(int n) {
    int result = 0;
    for (int i = 1; i <= ____n____; i++) {
        result = ____result + i____;
    }
    return result;
}
```

Looking at it more closely….

1 + 2 + 3 + 4 + … + n = n + (1 + 2 + 3 + 4 + … + (____n-1____))

We can say that sum(n) = n + sum(____n-1____).

Now, we can rewrite our solution differently.

```
public static int recursiveSum(int n) {
    if (n == ____1____) return n;          // base case
    return n + recursiveSum(____n-1____); // recursive case
}
```

Recursive definition

A definition contains a call to itself. Recursive programming has a method that calls itself.

A method call is a transfer of control to the start of the method. This can take place from within the method itself as well as from outside.

Base case

However, we do not want to call the method itself recursively to infinity (*infinite recursion*).

The condition that leads to a recursive method returning without making another recursive call is base case(s).

Every recursive method has a base case or base cases!

Step 1: Conceptual View

```
public static int recursiveSum(int n) {
    if (n == _____1_____) return n;      // base case
    return n + recursiveSum(___n-1_____); // recursive case
}
```

recursiveSum(5) 15
5+recursiveSum(4) 10+5 = 15
4+recursiveSum(3) 6+4 = 10
3+recursiveSum(2) 3+3 = 6
2+recursiveSum(1) = 1 1+2 = 3

Bookkeeping
For a recursive method, there is a lot of bookkeeping information that needs to be stored. Here, there are five pending calls till it hits its base case.

These pending calls may be deeply nested. All these pending calls are pushed onto the call stack. As the call hits the base case, then there is a return value and from that point, we pop the call from the stack to calculate and return a value.

The question is "is recursion efficient?"

Usually, recursion is used because it simplifies a problem conceptually, not because it is more efficient.

Can be more expensive

Example 1: Fibonacci Numbers

In 1170, Leonardo Pisano, who is also known as Fibonacci, was born in Pisa, Italy.

A sequence of numbers such that each number is the sum of the previous two numbers in the sequence, starting the sequence with 0 and 1.

Fibonacci Numbers

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55 …

Let's define this recursively.

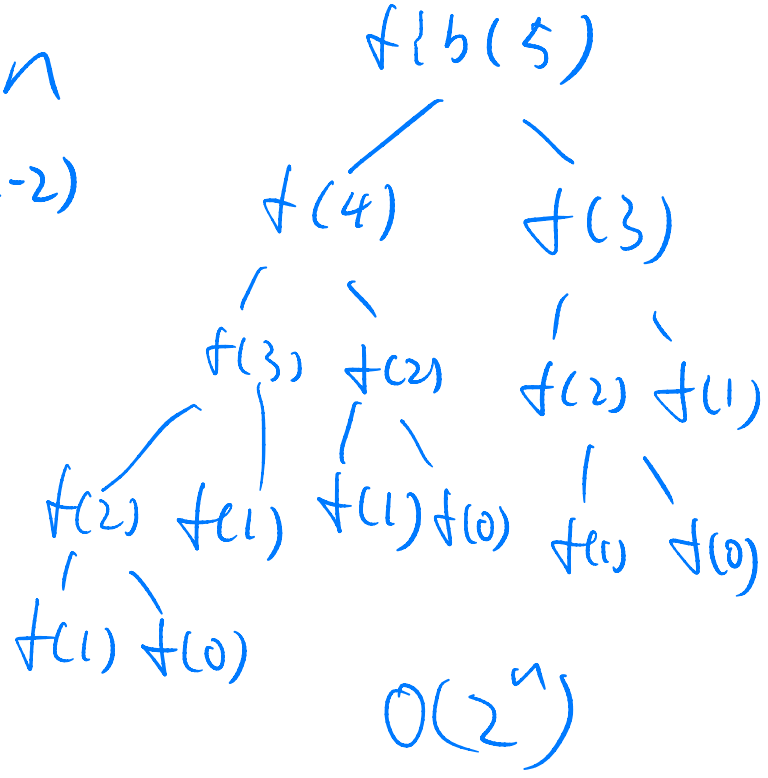
What is the base case(s)?

What is the recursive case?

Assuming that we are trying to calculate fibonacci(5), what would happen conceptually? (**recursion tree**)

If (n==0 || n==1) → n

fib(n-1) + fib(n-2)



Step 2: Implementation View
Remember the binary search we discussed in Module 4?

Our goal was to find a given value's location (index) in an **ordered** array using the fewest number of comparisons. The solution was to divide the array in half and see which half has the given value and do the same thing again and again till we find (or do not find) the value.

Here is the code that we implemented at that time.

```
public static int binarySearch(int[] data, int key) {
    int lowerBound = 0;
    int upperBound = data.length - 1;
    int mid;
    while (true) {
        // not found
        if (_____lb > ub_____) {
            return -1;
        }
        mid = _____(lb+ub) / 2_____;
        // found
        if (data[mid] == key) {
            return mid;
        }
        if (data[mid] < key) {
            lowerBound = _____mid+1_____;
        } else {
            upperBound = _____mid-1_____;
        }
    }
}
```

Let's take a look at the binary search again.

Do you see it has a recursive nature?

What is the base case(s)?

What is the recursive case?

```
private static int find(int[] data, int key, int lb,
                        int ub) {
    if (lb > ub) {
        return -1;
    }
    int mid = (lb+ub)/2;
    if (data[mid] == key) {
        return mid;
    }
    if (data[mid] < key) {
        return find(data, mid+1, ub);
    } else {
        return find(data, lb, mid-1);
    }
}
```

The method calls itself over and over but, each time with a smaller range of arrays than before till it hits one of the base cases.

If we were to make this to be a public method, it would look more complicated for users to specify lowerBound and upperBound. In fact, a user of this recursive method may not know what should be initial lowerBound and upperBound values.

To remove this kind of burden or confusion from users, we can provide the following public method that has an initial call of the private helper method in it.

```
public static int find(int[] data, int key) {
    return find(data, key, 0, data.length-1);
}
```

Summary: characteristics of Recursive Methods

- *It calls itself*
- *When it calls itself, it does so to solve a smaller version of the same problem.*
- *There is a base case (or base cases) that does not call itself anymore.*